

Prompt Life · v0.27.16

# Journey Visual Aid Consistency System

Implemented the v0.27.16 pass: shared visual-aid grammar, numbered/bulleted callout rendering, visual-aid audit tooling, priority marker repairs, documentation, reports, and screenshot QA.

## What Changed

- Added shared visual-aid frame, diagram shell, caption, and callout-list grammar.
- Made numbered callouts appear only when marker counts match.
- Converted generated/atmospheric aids to bullet callouts.
- Repaired priority diagrams: Overfitting, Tokenization, Token IDs, Autoregression, Context Window, Grounding, and Hallucination.
- Bumped app/package version to `0.27.16`.

## Audit Summary

- Visual aids audited: 39.
- Generated PNG-backed aids: 4.
- Coded SVG/HTML aids: 35.
- Final marker/list mismatches: 0.
- Remaining redesign backlog: atmospheric Image 2 candidates and future precision polish, not blocking v0.27.16.

## Artifacts

- `docs/journey/visual-aids/VISUAL_AID_STYLE_GUIDE.md`
- `docs/journey/visual-aids/image-2-visual-prompts-v0-27-16.md`
- `docs/journey/visual-aids/visual-aid-consistency-audit-v0-27-16.json`, `.csv`, and `.md`
- `docs/journey/visual-aids/prompt-life-v0-27-16-visual-aid-consistency-report.html` and `.pdf`

## Verification

- `npm run typecheck`: passed.
- `npm run build`: passed; Vite large-chunk warning remains informational.
- `npm run build:pages`: passed; same large-chunk warning.

- `npm run audit:answers` : passed.
- `npm run audit:checkpoints` : passed.
- `npm run audit:question-clues` : completed; informational counts remain 36 term-echo candidates and 43 answer-length warnings.
- `npm run audit:learner-copy` : passed.
- `npm run audit:visual-aids` : passed, 39 aids, 0 mismatches.

## Browser QA

- 320px and 390px visual-aid checks showed no horizontal overflow.
- Overfitting, Tokenization, Vectors, Fine-Tuning PNG, generated PNG, and Inference coded SVG screenshots captured.
- Desktop overfitting sample captured.
- All eight compact stage links opened the expected first card, stayed at top scroll, and exposed Stage aria labels.

Screenshots

OVERFITTING-GENERALIZATION

# Overfitting vs Generalization

Lesson: Overfitting and Generalization

Pattern: overfitting

Variant: zen-garden-map

Callouts: numbered

Markers: 4

## Overfitting vs Generalization

Memorizing training examples is not the same as learning patterns that transfer to set-aside validation examples.

1 **Training examples** Old dots are examples the model fit during training.

2 **Validation examples** Set-aside examples are kept out of training to test transfer.

3 **Overfit curve** The overfit curve traces old dots too tightly.

4 **Generalizing curve** The smoother curve reaches new examples better.

**Key takeaway:**  
A model should learn transferable patterns, not just memorize the training dots.

LEARNING OBJECTIVE

Memorizing training examples is not the same as learning patterns that transfer to set-aside

Overfitting numbered markers at 390px

TOKENIZATION

# Text to Tokens

Lesson: Tokenization

Pattern: token

Variant: paper-diagram

Callouts: numbered

Markers: 3

## Text to Tokens

Uneven chunks, punctuation included

Text is split into model-readable chunks before token IDs and embedding lookup.

1 **Simplified split** A | jealous | dog | chased | a | startled | cat | across | the | kitchen | floor | .

2 **Uneven chunks** Some

Tokenization badges at 320px

file:///Users/kevinhegg/Desktop/promptlife/docs/journey/visual-aids/prompt-life-v0-27-16-final-response-with-screenshots.html

3/5

## VECTORS

## Feature Vector

Lesson: Vectors

Pattern: bars

Variant: origami-object

Callouts: numbered

Markers: 3



## Feature Vector

## Teaching labels versus distributed features

A vector is a list of numbers; real features are distributed rather than neat English sliders.

- Teaching sliders** Labels such as animal-ish or tone are simplified for learning.
- Distributed features** Real model features are spread across many dimensions.
- Why vectors** Vectors let the model compute with fuzzy numerical relationships.

## Key takeaway:

Vectors are useful because many fuzzy features can vary at once.

## LEARNING OBJECTIVE

Let learners compare a simplified slider view with unlabeled distributed numerical features.

## ACCESSIBLE DESCRIPTION

Vectors numbered markers at 390px

## BEFORE-MORNING-LLM-CLOUD

## Prompt to Prediction

Lesson: What Is an LLM?

Pattern: generatedImage

Variant: generated-image

Callouts: bullets

Markers: 0

Asset: before-morning-llm-cloud.png

Type: generated-image



## Prompt to Prediction

## Score, choose, append, repeat

Current context enters the model, learned weights shape next-token probabilities, one token is generated, and the response grows token by token.

- Context enters** The current prompt and response-so-far become the visible input for this run.
- Weights shape probabilities** Learned weights, built earlier during training, shape next-token scores.
- One token is generated** The model chooses

Generated PNG with bullet callouts at 390px



## INFERENCE-PASS

## Forward Pass

Lesson: Inference

Pattern: inference

Variant: neon-flow

Callouts: numbered

Markers: 4



## Forward Pass

## Fixed weights, temporary activations

Inference uses fixed weights to create temporary activations, hidden states, and next-token scores.

- 1 **Context** The current prompt, retrieved text, conversation, and response-so-far enter the run.
- 2 **Fixed weights** The learned weights are used but not normally updated.
- 3 **Temporary activations** The run creates temporary internal states that lead to next-token scores.
- 4 **Next token** Decoding chooses one response token, then the loop can repeat.

## Key takeaway:

Inference is a forward pass, not durable training.

## LEARNING OBJECTIVE

Inference coded SVG at 390px

## OVERFITTING-GENERALIZATION

## Overfitting vs Generalization

Lesson: Overfitting and Generalization

Pattern: overfitting

Variant: zen-garden-map

Callouts: numbered

Markers: 4



## Overfitting vs Generalization

Memorizing training examples is not the same as learning patterns that transfer to set-aside validation examples.

- 1 **Training examples** Old dots are examples the model fit during training.
- 2 **Validation examples** Set-aside examples are kept out of training to test transfer.
- 3 **Overfit curve** The overfit curve traces old dots too tightly.
- 4 **Generalizing curve** The smoother curve reaches new examples better.

**Key takeaway:**  
A model should learn transferable patterns, not just memorize the training dots.

## LEARNING OBJECTIVE

Overfitting desktop sample

Known issue: no blocking known issues for v0.27.16. The next visual pass should decide which atmospheric concepts get Image 2 assets and keep exact mechanics coded.